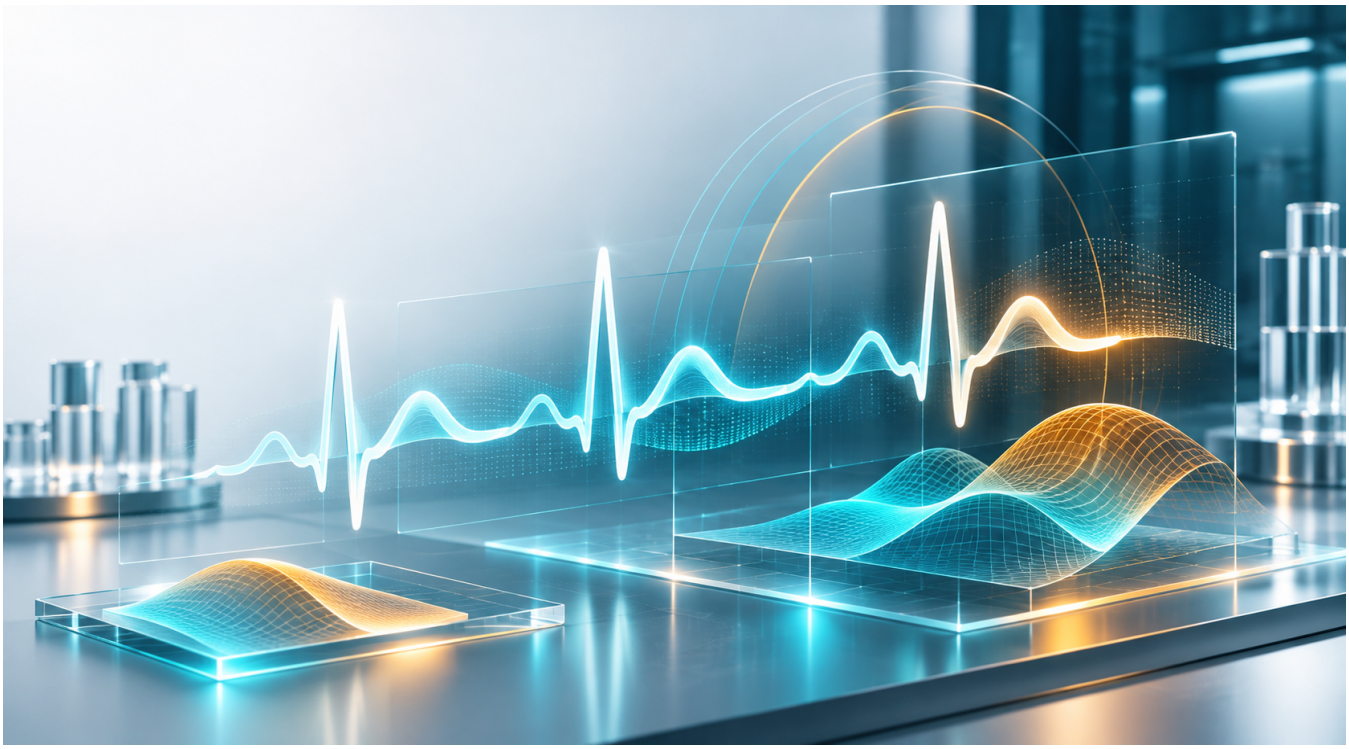
ECG Feature and Rule Evaluation

Reusable mathematical ECG postprocessing with paceval



Research outcome

An original paceval C++ integration evaluated 51 persistent mathematical graphs over 10,000 deterministic synthetic feature records, with independent native checks and a separately implemented 80-decimal-digit audit.

Execution graphs	50 distinct models + 1 immutable revision
Scalar evaluations	13,923
Batch comparisons	1,600
Independent high-precision checks	500 passed
Scientific figures	Eight result graphics from paceval-generated CSV

Scope: Non-clinical mathematical research; no patient recordings, diagnostic classification, clinical sensitivity/specificity, device validation or prospective evaluation.

1. Industrial context and research question

Ambulatory ECG monitors, ECG interpretation pipelines and biomedical software work with characteristic waveform measurements such as successive RR intervals, QRS-complex duration and signal-amplitude features. A pipeline may compute these features first, then apply configurable scoring and review rules to previously measured values.

Current state of the art

Established ECG research uses annotated waveform corpora and standardized preprocessing. The PhysioNet MIT-BIH database provides 48 approximately half-hour, two-channel recordings, while PTB-XL contains 12-lead ECG recordings for research. The LUDB collection provides delineation annotations for 200 ten-second 12-lead recordings. These datasets support research but do not make arbitrary score coefficients medically meaningful.

Industrial question

Can a manufacturer or laboratory maintain multiple explicit feature-to-score rules as separately versioned mathematical formulas, evaluate them repeatedly on standardized feature vectors, and reproduce their output after formulas change?

System boundary

Upstream host: ECG acquisition, filtering, detection, beat delineation, extraction, normalization and data-governance policy. paceval: expression creation, persistent handles, scalar evaluation and independent-value-set batch evaluation. Downstream host: threshold flagging, storage and review workflow.

Important: The research uses no real ECG waveforms and does not calculate clinically validated RR or QRS measurements. Its inputs have plausible illustrative units but fully synthetic values.

2. Defined mathematical feature model

The illustrative source expression in WP032 is score = a RRmean + b RRstd + c QRS_width. This implementation extends that concept to normalized features so all sums and nonlinear terms are dimensionless.

r	(RRmean - 800 ms) / 100 ms
d	(RRstd - 15 ms) / 20 ms
q	(QRSwidth - 90 ms) / 15 ms
a	(Amplitude - 1 mV) / 0.2 mV

Five score families

Every family contains ten distinct alpha/beta coefficient settings (50 separate expression strings):

F1 Linear: z = alpha*r + beta*d + 0.36*q + 0.12*a

F2 Interaction: z = alpha*r*d + beta*q*a + 0.19*d

F3 Quadratic: z = alpha*r^2 + beta*d^2 + 0.22*q^2 - 0.3

F4 Periodic: z = alpha*sin(r) + beta*cos(q) + 0.17*d*a

F5 Log-exp: z = alpha*ln(1+r^2) + beta*exp(-d^2) + 0.18*q*a

Common nonlinear transform

score = 1 / (1 + exp(-z)). This bounds the score between zero and one but does not convert it into a calibrated probability of arrhythmia or any other medical condition.

Coefficients: alpha = 0.28 + 0.048 j + 0.015 f; beta = 0.16 + 0.032 j, with family f = 0..4 and variant j = 0..9. The version-2 copy of model 1 increases alpha by 0.09 without changing model 1.

3. paceval execution architecture

Verified API boundary

The C++ executable compiles the supplied original paceval_main.cpp, includes paceval_main.h, and links against a compatible external Linux paceval 4.25 runtime. The shared library is intentionally not part of the public ZIP.

Initialization	paceval_InitializeLibrary
Expression compilation	paceval_CreateComputation
Scalar computation	paceval_dGetComputationResult
Multi-input batch	paceval_dGetComputationResultExt
Error inspection	paceval_GetIsError and per-batch error codes
Cleanup	paceval_DeleteComputation; paceval_FreeLibrary

Controlled versioning and feature flow

Fifty formula handles are created at startup and reused with changed feature inputs. A revised model creates a 51st handle; the original remains callable and yields its original values. The C++ host handles feature unit conversion, validity checks, deterministic test generation and review thresholding.

Batch and scalar comparison

Each of the 50 graph handles was evaluated on 32 independent deterministic feature vectors using the official batch API; 1,600 batch values were checked by a separately coded C++ mathematical reference.

This experiment tests reusable expression execution, not integrated ECG acquisition, real-time latency, diagnostic classification or medical-device qualification.

4. Deterministic workload and completed checks

A seeded xorshift64* generator produces 10,000 synthetic feature inputs. Family assignment cycles through all 50 formulas. Every output is independently evaluated in native C++ using identical mathematical meaning. Separately, 500 exported cases were checked using 80-digit mpmath operations.

Seed	20261112
Synthetic replay feature records	10,000
Persistent formula handles	51
Scalar evaluations	13,923
Independent batch checks	1,600
High-precision spot checks	500 / 500 passed
CMake and CTest	1 / 1 passed

Non-clinical review policy

The demonstration host flags a score if it is at least 0.65. In this fixed synthetic workload, 1,658 records meet that policy. The number is a deterministic demonstration result, not a false-alarm rate, patient-risk estimate or clinical screening performance metric.

Validation artifacts

The full numerical comparison remains in results/ technical logs and high-precision audit JSON. The public result figures and discussion focus on computed feature scores, version changes and the effect of rule selection rather than numerical deviations between C++ and paceval.

5. Feature-score replay and family behaviour

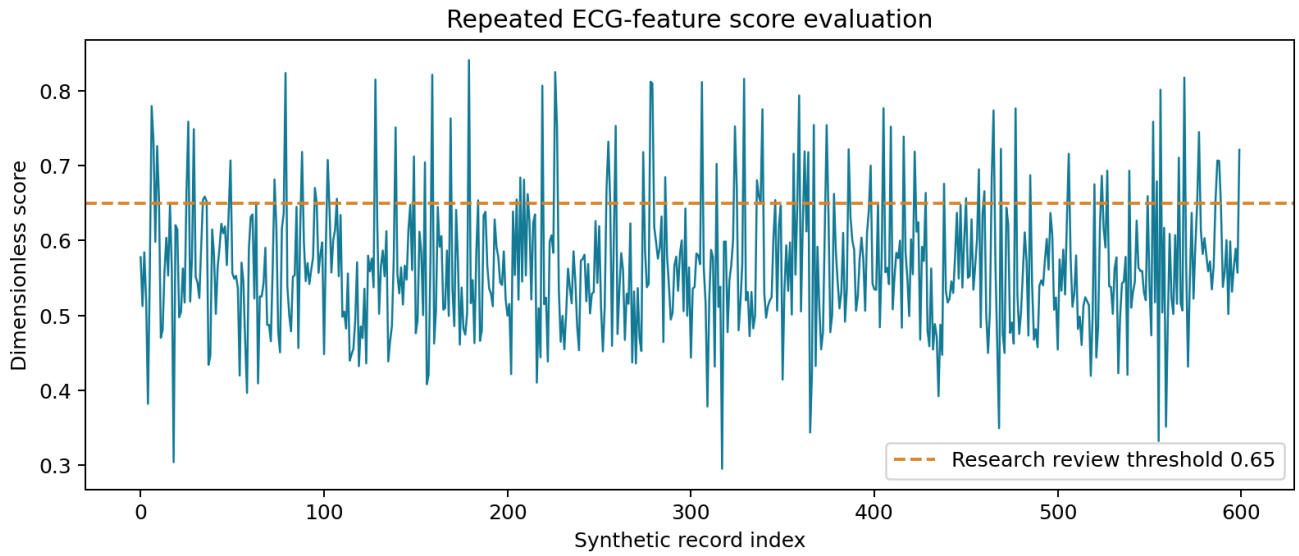


Figure 1. The first 600 paceval-derived score values; the dashed threshold is a deliberately synthetic review policy.

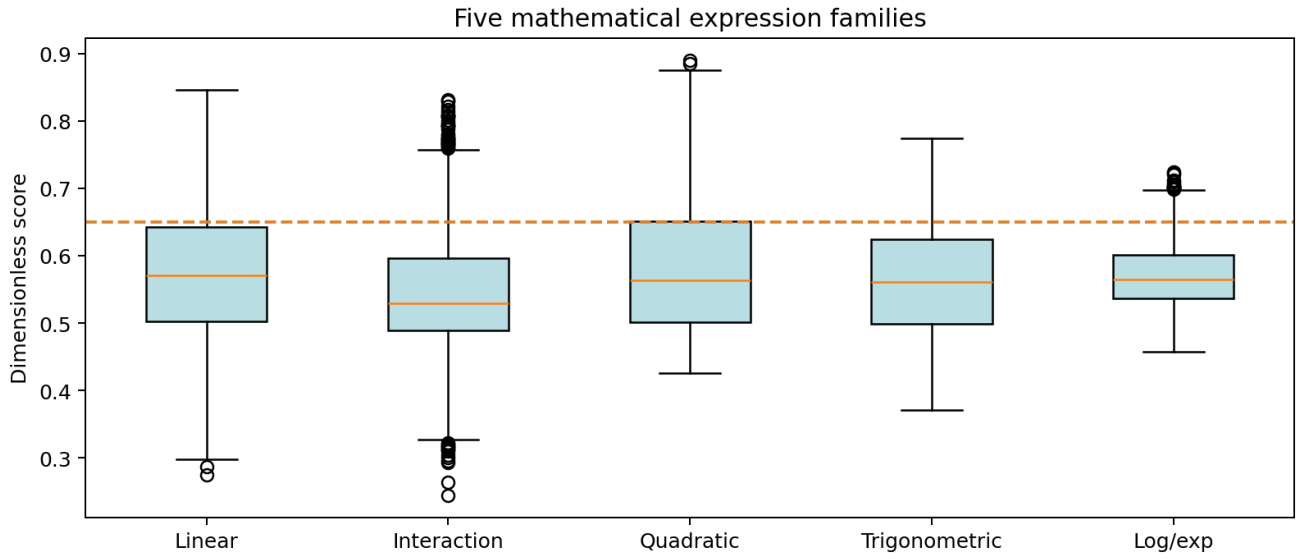


Figure 2. Score distributions by expression family over the synthetic replay. The differences reflect formula design, not diagnostic merit.

6. Feature score response surfaces

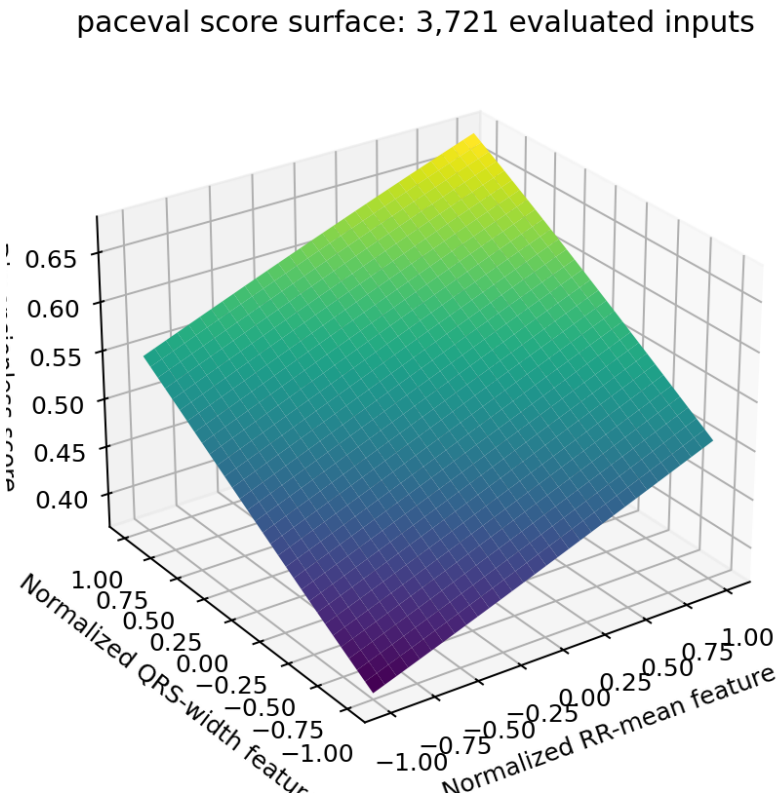


Figure 3. An actual 61 x 61 paceval-evaluated score surface at fixed dispersion and amplitude features.

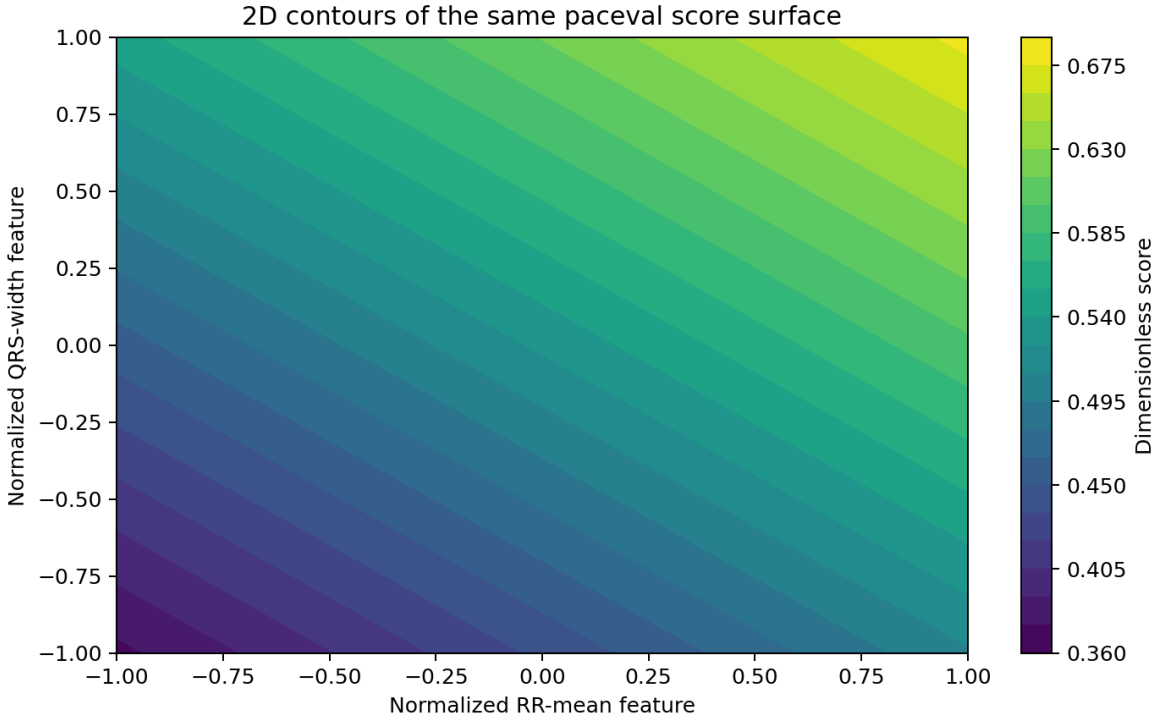


Figure 4. Contours of the same 3,721 score evaluations, showing sensitivity to the normalized RR-mean and QRS-width inputs.

7. Versioning and review-rule sensitivity

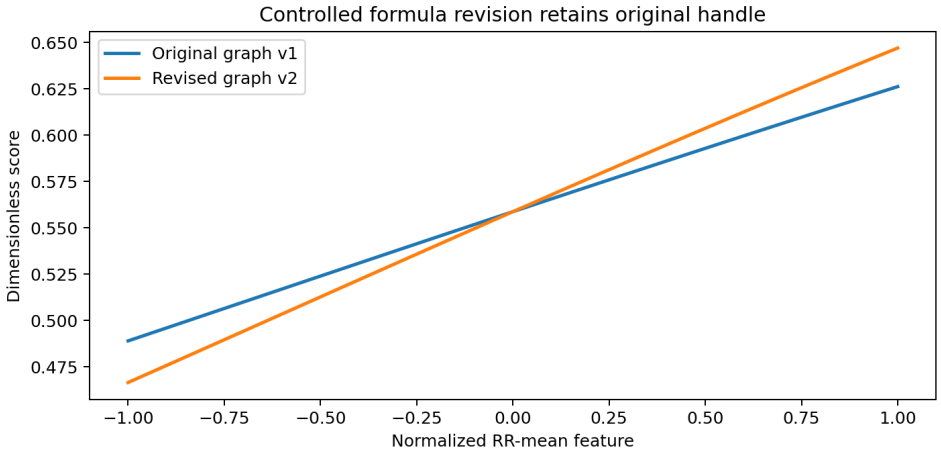


Figure 5. Both formula versions remain simultaneously callable using separate persistent handles.

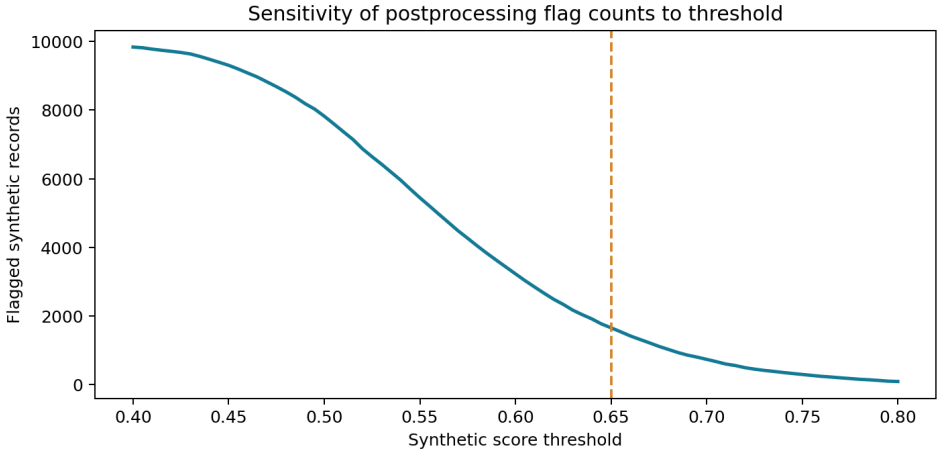


Figure 6. The number of synthetic records flagged varies with the host-chosen review threshold; no clinical operating point is claimed.

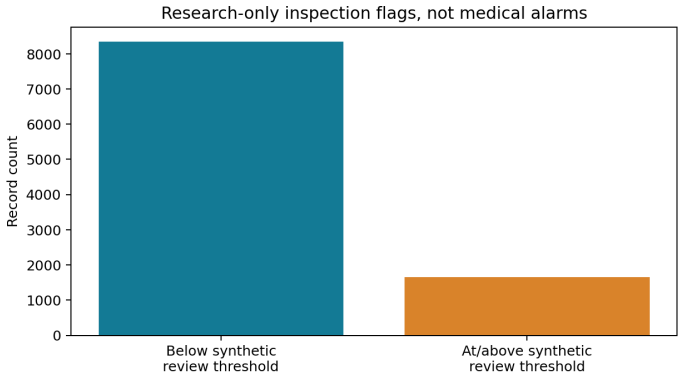


Figure 7. Fixed replay summary under the illustrative 0.65 research threshold.

8. Interpretation, limitations and next stage

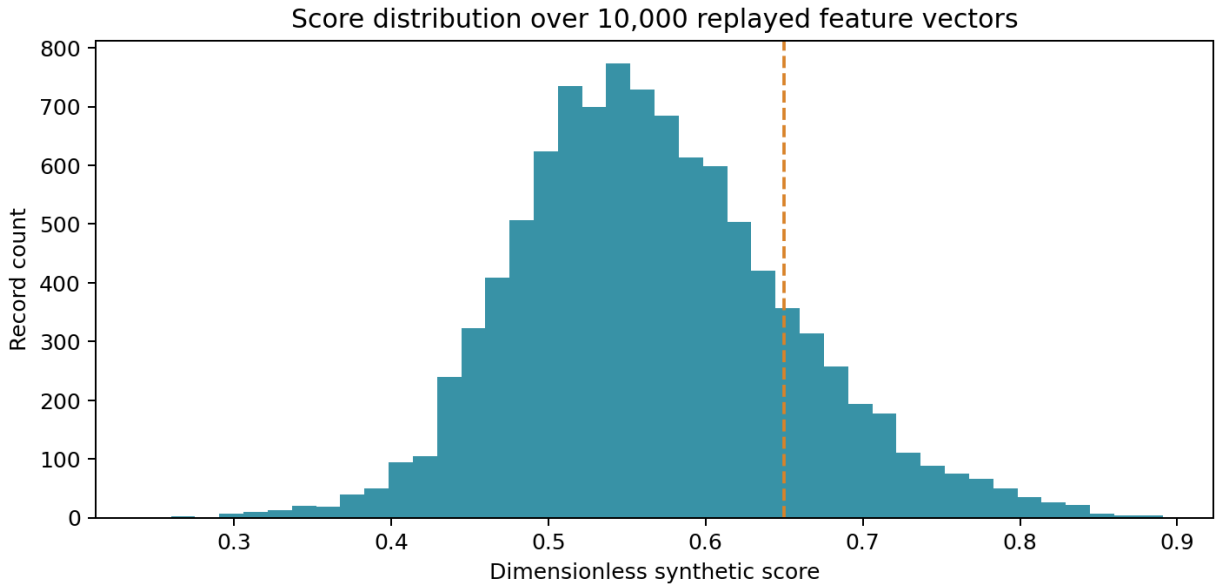


Figure 8. Computed score frequencies across the 10,000 synthetic, non-patient feature vectors.

What has been demonstrated

The defined mathematical features-to-score pipeline can use reusable paceval computation handles, multiple distinct formula families, batch evaluation, and controlled model revision. Numerical output passed the separately implemented reference checks for this deterministic workload.

What has not been demonstrated

The demonstration has no empirical ECG waveform processing, validated beat detection, expert-labeled evaluation, patient stratification, external validation cohort, calibrated clinical risk probabilities, diagnostic sensitivity or specificity, prospective study, or medical-device clearance.

Meaningful future experiment

With appropriate permissions and license review, an independent host could extract fixed, audited RR and QRS features from PhysioNet records, preserve record-level train/test splits, and compare only reproducible mathematical postprocessing. Any diagnostic study would require a clinical protocol and separate validation.

9. Reproducibility, sources and conclusions

Rebuild and run

cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/abs/path/libpaceval.so

cmake --build build -j

ctest --test-dir build --output-on-failure

./build/ecg_rules --records 10000 --out examples

python3 src/high_precision_reference.py examples

python3 src/make_figures.py ; python3 src/make_whitepaper.py

python3 src/check_readme_math.py

Primary data files

examples/feature_replay.csv; examples/formula_registry.csv; examples/ecg_score_surface.csv; examples/formula_version_surface.csv; examples/execution_summary.txt. Technical audits are in results/. All eight scientific figures are regenerated from those actual paceval-exported CSV values.

Scientific and technical sources

PhysioNet, MIT-BIH Arrhythmia Database (v1.0.0): https://physionet.org/content/mitdb/1.0.0/

PhysioNet, PTB-XL ECG Database: https://physionet.org/content/ptb-xl/

PhysioNet, LUDB ECG delineation data: https://physionet.org/content/ludb/1.0.1/

paceval Mathematical Engine C/C++ API v4.25: https://paceval.com/api/

Conclusion

The research establishes a reproducible and independently checked mathematical execution demonstrator for configurable ECG-feature rules. It is a non-clinical software-engineering feasibility study, not evidence of ECG diagnostic performance or clinical safety.